

UNIVERSAL SUBSTRATE THEORY

PLATINUM DOCUMENT #1

Formal Variables and Update Law Specification

*The Minimal State Vector, Update Operators, Memory Kernel, Recoverability Function,
Compatibility Metric, and Threshold Surfaces*

This document defines the first disciplined formal machinery required for Universal Substrate Theory to move from architectural language into simulation-ready specification. It does not claim completion. It freezes a minimal variable set, proposes provisional mathematical objects, identifies update rules, and states the open debt that must be paid before correspondence to known physics can be treated as more than a controlled research target.

Prepared as a restrained technical specification for the UST formalisation programme

Version: Platinum Document #1 | Status: Minimal formal scaffold | Scope: simulation-first

0. Purpose and Scientific Posture

0.1 Purpose of the Document

The purpose of this document is to define the formal objects that a first UST simulation can manipulate without relying on metaphor, hidden geometry, or loose interpretive language. It establishes a minimal substrate state vector, a family of update operators, a compatibility metric, a memory kernel, a recoverability function, a loading function, and a set of threshold surfaces. These are not yet final laws of nature. They are the first lawful scaffolding for disciplined formal exploration.

The immediate value of this document is operational. Once the variables and update order are explicit, UST can be tested as a toy formal system. That toy system can then be judged on internal stability, reproducibility, regime behaviour, and possible correspondence with known physical domains.

Core posture: UST is not advanced by claiming a final equation too early. It is advanced by freezing a minimal algebra, running it honestly, identifying where it fails, and only then strengthening the formal layer.

0.2 What This Document Is Not

This document is not a complete physical theory. It does not replace general relativity, quantum mechanics, quantum field theory, or the Standard Model. It does not claim a derivation of known physics. It does not define a hidden-variable model. It does not assert that the proposed functions are unique or physically final. Its role is narrower: to specify the first candidate formal machinery required for simulation and correspondence discipline.

0.3 Scientific Posture

The scientific posture is regime-bound, conservative, and explicit about unknowns. The architecture comes before equations, but equations are now required to prevent the architecture from becoming unfalsifiable. The correct posture is not inflationary certainty. It is formal humility: define, simulate, compare, revise.

- Regime-bound validity: each equation or operator applies only inside a stated domain.
- Architecture before equations: the variables must express the ontology before optimisation begins.
- Explicit unknowns: every unspecified function is named as debt rather than hidden by language.
- Simulation-first formalisation: early value comes from reproducible toy systems.
- Correspondence discipline: known physics is approached by mapping regimes, not by renaming terms.

1. The Minimal Substrate State Vector

1.1 Motivation

A minimal state vector is required because a theory cannot be simulated unless its local degrees of freedom are fixed. UST currently uses several recurring conceptual variables: organisation, memory, recoverability, loading, compatibility, and coherence capacity. These variables must now be treated as formal components rather than descriptive labels.

The state vector is the minimum object needed for threshold definition, update ordering, regime mapping, and falsifiability. It allows a simulation to ask whether a local substrate element remains viable, whether it joins an envelope, whether it accumulates irreversible memory, whether recoverability declines, and whether a regime transition occurs.

1.2 Canonical State Vector

$$s_i(t) = (o_i(t), m_i(t), r_i(t), l_i(t), c_i(t), q_i(t))$$

Each local substrate element i carries six variables at update step t . The notation does not require the element to be a point in space. In the minimal interpretation, i indexes a relational site, update cell, graph node, or abstract substrate unit. Later versions may replace i with an equivalence class, envelope node, or fibre-like relational object.

The components are defined as follows. o_i is the organisation state, representing local structural arrangement. m_i is memory load, representing accumulated history-dependent constraint. r_i is recoverability, representing the local capacity to re-lock after disturbance. l_i is loading or strain, representing deformation demand. c_i is compatibility status, representing relational fit with neighbouring or envelope-linked states. q_i is coherence capacity, representing the ability to support multi-allocation, envelope stability, or non-classical coupling.

1.3 State Vector Domain

The first simulation should use normalised bounded domains unless a specific variable requires tensor or vector extension. A simple starting convention is:

$$o_i \text{ in } O, \quad m_i \text{ in } [0,1], \quad r_i \text{ in } [0,1], \quad l_i \text{ in } [0,1], \quad c_i \text{ in } [0,1], \quad q_i \text{ in } [0,1]$$

Here O may be discrete, continuous, or categorical. The first toy system can set $O = \{0, 1, 2, \dots, K\}$ for a finite organisation class. Later systems may define o_i as a vector, graph motif, local ordering tensor, or mode descriptor. The bounded scalar variables are not claims that the substrate is literally normalised. They are simulation conventions for the first formal stage.

Boundary conditions must also be explicit. The initial toy model should allow three boundary modes: closed finite graph, open finite graph with damping, and periodic graph. Each mode must be recorded because thresholds and collapse-like transitions may depend strongly on boundary behaviour.

1.4 State Vector Interpretation

The state vector is pre-geometric. It is not a classical field on spacetime, not a hidden particle state, and not a direct replacement for metric geometry. Geometry is treated as a possible emergent translation of recoverability, compatibility, pacing, and threshold stability under specific low-load regimes. The vector is therefore relational before it is spatial.

2. Update Operators: The Substrate Update Algebra

2.1 Overview

The update algebra defines how substrate states change from one tick to the next. The first goal is not to identify the final operator of reality, but to specify a lawful family of operators that can be simulated and compared. The general form is:

$$s_i(t+1) = U_i[s_i(t), N_i(t), W_i(t), M_i(t), R_i(t), \text{Theta}(t)]$$

N_i is the relational neighbourhood of i . W_i contains compatibility weights. M_i contains memory contributions. R_i contains recoverability constraints. Theta denotes threshold surfaces or regime gates. The operator U_i must be local or quasi-local in the substrate sense, not necessarily spatially local in an already given geometry.

2.2 Local Update Operator

The local operator updates a site from its internal state alone. It is useful for testing memory accumulation, recoverability decay, and internal relaxation without relational effects.

$$s_i^{\text{local}}(t+1) = U_L[s_i(t)]$$

A minimal form can be built component by component. Organisation tends toward local stability, memory accumulates under change, recoverability relaxes when load is low and declines when load is high, loading decays or grows depending on imposed strain, compatibility is provisionally unchanged, and coherence capacity decays under memory or loading pressure.

2.3 Quasi-Local Update Operator

The quasi-local operator introduces compatibility-weighted influence from relational neighbours. It is the first place where UST becomes substrate-like rather than cell-automaton-like. Neighbourhood is defined by compatibility, shared envelope membership, or adjacency in the abstract update graph.

$$s_i^{\text{ql}}(t+1) = U_Q[s_i(t), \{s_j(t)\} \text{ for } j \text{ in } N_i, w_{ij}(t)]$$

The weights w_{ij} should be derived from a compatibility metric rather than assigned arbitrarily. In the first toy model, w_{ij} may be normalised so that the total incoming influence to each site is bounded.

2.4 Distributed Update Operator

The distributed operator acts on a cluster or envelope rather than on a single site. This is necessary for modelling coherence, multi-allocation, envelope formation, and collapse-like coordinated transitions.

$$S_E(t+1) = U_D[S_E(t), B_E(t), \Theta_E(t)]$$

S_E is the aggregate state of envelope E . B_E denotes boundary pressure or leakage at the envelope edge. Θ_E denotes threshold surfaces evaluated at the envelope level. Distributed update cannot be treated as ordinary averaging, because coherence capacity and compatibility topology may change the available update paths.

2.5 Memory-Weighted Update

Memory must affect future transitions. A simple first rule is additive accumulation with decay, saturation, and event sensitivity:

$$m_i(t+1) = \text{sat}[\lambda_m m_i(t) + \alpha_m \Delta_i(t) + \beta_m \sum_j w_{ij}(t) \Delta_j(t)]$$

Here λ_m is the memory persistence factor, Δ_i is local update disturbance, and sat is a bounded saturation function. The term involving neighbours allows memory to propagate through compatible structures without treating it as a field already living in spacetime.

2.6 Recoverability-Weighted Update

Recoverability modifies update speed, stability, and viability. Low recoverability should slow re-locking, increase threshold sensitivity, and make collapse-directed update more likely. A first scalar rule can be written as:

$$r_i(t+1) = \text{clamp}[r_i(t) + \gamma_r(1-l_i(t)) - \delta_r l_i(t) - \epsilon_r m_i(t)]$$

This is not final physics. It is an interpretable prototype: recoverability improves when load is low, declines when load is high, and is further reduced by accumulated memory burden.

2.7 Update Sequencing

The minimal simulation must freeze update order. The recommended first order is: compute compatibility weights, compute local disturbance, update memory, update loading, update recoverability, update coherence capacity, update organisation, then evaluate thresholds. This order is provisional but must be fixed for reproducibility.

Update sequencing is not a cosmetic detail. If thresholds are evaluated before memory or recoverability, the same initial state may enter a different regime. Every run must log its update order.

2.8 Update Failure Modes

UST requires explicit update failure modes because regime transitions are not merely smooth changes in variable values. A skipped update represents failure to complete a tick. A saturated update represents a variable reaching a bounded ceiling or floor. A fragmented update represents loss of envelope coherence or compatibility. A collapse-directed update represents monotonic movement toward a lower-recoverability basin.

3. Compatibility Metric

3.1 Definition

Compatibility is a non-spatial relational metric measuring whether two substrate states can exchange update influence, cohere inside an envelope, or re-lock into a shared structure. It should not be confused with physical distance. It is closer to relational viability than separation.

3.2 Metric Structure

$$D_{ij} = a_o d_o(o_i, o_j) + a_m |m_i - m_j| + a_r |r_i - r_j| + a_l |l_i - l_j| + a_q |q_i - q_j|$$

$$C_{ij} = \exp(-D_{ij} / \sigma_c)$$

D_{ij} is a weighted incompatibility distance. C_{ij} is a bounded compatibility score. The constants a_o , a_m , a_r , a_l , and a_q must be logged. The scale σ_c controls how quickly compatibility decays with state difference.

3.3 Components

Structural distance compares organisation states. Memory distance compares accumulated history. Recoverability distance compares re-locking capacity. Loading distance compares strain state. Coherence distance compares envelope-support capacity. These components can later be expanded into vectors or tensors, but the first toy model should keep them scalar unless a specific experiment requires otherwise.

3.4 Compatibility Clustering

Clusters form where compatibility remains above a chosen viability threshold. An envelope can be defined as a connected set of sites with sufficient internal compatibility, sufficient coherence capacity, and bounded loading divergence.

$$E = \{i : C_{ij} \geq \kappa_E \text{ for connected } j, q_i \geq q_E, l_i \leq l_E\}$$

This definition is intentionally minimal. It allows a simulation to detect envelope formation without assuming in advance that the envelope is a particle, field mode, or geometric region.

3.5 Compatibility Failure

Compatibility failure occurs when the relational distance between sites exceeds the range that can support coherent update. This may generate fragmentation, envelope rupture, incompatible re-locking, or decoherence-

like behaviour. In a strong-field interpretation, compatibility failure may contribute to basin entry when recoverability is also low.

4. Memory Kernel

4.1 Purpose

Memory is the substrate's history-dependent constraint. It records that the current state is not only determined by instantaneous variables but also by the path taken to reach them. Without a memory law, UST cannot represent hysteresis, irreversible collapse, decoherence history, or threshold path dependence.

4.2 Kernel Structure

A first memory kernel should combine additive accumulation, decay, saturation, and hysteresis. The simplest continuous form is:

$$M_i(t) = \sum_{\tau=0}^t K(t-\tau) \Delta_i(\tau), \quad K(\tau) = A \exp(-\tau/\tau_m)$$

For discrete simulation, the recursive rule in section 2.5 is easier to implement. The kernel form is useful because it makes clear that memory is not just a value. It is a weighted record of past disturbance.

4.3 Memory Propagation

Memory can be local, distributed, or envelope-linked. Local memory is stored at a site. Distributed memory is shared across a cluster through compatibility-weighted propagation. Envelope-linked memory persists as a collective constraint, meaning the envelope's future update paths are shaped by its past transitions.

4.4 Memory Failure

Memory failure does not mean forgetting. In UST, it means memory burden becomes so high that previous re-locking routes become unavailable. This can generate irreversibility, collapse susceptibility, decoherence persistence, or basin trapping. The simulation should track both memory value and memory growth rate because rapid accumulation may be more destabilising than high stable memory.

5. Recoverability Function

5.1 Definition

Recoverability is the substrate's ability to re-lock after disturbance. It is the central bridge between UST's formal variables and possible correspondence with curvature, inertia, time dilation, and strong-field collapse. A highly recoverable state returns to viable update structure after perturbation. A low-recoverability state cannot easily re-establish stable relational order.

5.2 Formal Structure

A provisional recoverability function can depend on organisation stability, memory burden, loading, compatibility, and coherence capacity:

$$R_i = R(o_i, m_i, l_i, c_i, q_i) = \rho_o o_i + \rho_c c_i + \rho_q q_i - \rho_l l_i - \rho_m m_i$$

$$r_i = \text{sigma}(R_i) = 1 / (1 + \exp(-R_i))$$

The first line gives a linear score. The second maps the score into a bounded recoverability value. This is not proposed as a final physical law. It is a first testable structure for simulation, where parameter sensitivity can be measured.

5.3 Local vs Distributed Recoverability

Local recoverability belongs to a site. Envelope-level recoverability belongs to a coherent cluster. Cluster-level recoverability belongs to a larger relational basin or regime. These levels should not be conflated. A local site may be weak while an envelope remains stable, and an envelope may be stable while the larger cluster is approaching saturation.

5.4 Recoverability Gradients

Recoverability gradients are the most important candidate bridge to known physics. A gradient in recoverability may translate into effective curvature under weak-field conditions. Resistance to change may translate into inertia. Slower update sequencing in low-recoverability regions may translate into time dilation. These are correspondence hypotheses, not established derivations.

Working translation hypothesis: curvature, inertia, and time dilation are not inserted into UST as geometric primitives. They are candidate low-regime descriptions of recoverability gradients, update pacing differences, and compatibility-constrained propagation.

5.5 Recoverability Failure

Recoverability failure occurs when a state cannot re-lock into the previous viable update pattern. In weak regimes this may appear as instability or decoherence. In strong-field regimes it may correspond to collapse-directed behaviour. In the black-hole replacement programme, singularity is not treated as a point of infinite density but as entry into a deformity basin where ordinary geometric translation fails.

6. Loading and Strain Function

6.1 Definition

Loading is deformation demand on the substrate. It measures how much strain a state or envelope must carry to maintain its update structure. Loading is not identical to mass, energy, stress, or curvature, but it may correspond to combinations of them in specific regimes.

6.2 Formal Structure

$$l_i(t+1) = \text{clamp}[\lambda_l l_i(t) + S_i(t) + \eta_l \sum_j w_{ij}(t)(l_j(t) - l_i(t))]$$

S_i is an imposed source or internally generated strain term. The neighbour term permits loading diffusion or redistribution across compatible links. The parameter λ_l controls persistence. This structure lets a simulation explore local strain, distributed strain, and mode saturation.

6.3 Loading-Recoverability Interaction

High loading should reduce recoverability and increase memory burden. This link is essential. Without it, collapse thresholds become arbitrary. The minimal coupling is:

$$\partial r_i / \partial l_i < 0, \quad \partial m_i / \partial l_i > 0$$

The first inequality states that greater load reduces recoverability. The second states that greater load increases memory accumulation. This is a core rule for the first formal UST toy system.

7. Threshold Surfaces

7.1 Purpose

Threshold surfaces replace single-value thresholds and help prevent singularity language from re-entering the theory. A regime transition should occur when a function of state variables crosses a boundary, not when one scalar reaches a mystical value. Thresholds must therefore be surfaces in state space.

7.2 Formal Definition

$$T_k(s_i, E, t) = F_k(o_i, m_i, r_i, l_i, c_i, q_i, \text{grad}_C, \text{grad}_R, M_E) = 0$$

Each threshold T_k is defined by a function F_k over local state variables, compatibility gradients, recoverability gradients, and optionally envelope memory M_E . A transition occurs when the sign, distance, or stability around this surface changes.

7.3 Major Thresholds

The first formal programme should define at least five threshold classes. Geometric viability marks the region where stable locality and metric-like translation remain possible. Quantum coherence marks the region where envelope or multi-allocation structure is supported. Decoherence boundary marks loss of envelope stability into classical-like separability. Saturation boundary marks the onset of overloaded update structure. Collapse boundary marks entry into a low-recoverability basin.

$$T_{\text{geo}}, T_q, T_{\text{dec}}, T_{\text{sat}}, T_{\text{col}}$$

7.4 Threshold Hysteresis

Thresholds must be path-dependent where memory matters. Crossing out of a regime should not always be equivalent to crossing back into it. This creates hysteresis and allows irreversible behaviour without inserting irreversibility by hand.

$$T_k^{\text{in}}(s, m) \neq T_k^{\text{out}}(s, m)$$

This asymmetry is especially important for collapse, decoherence, and strong-field saturation. A system that has accumulated memory may not regain earlier coherence or recoverability simply by reversing the loading variable.

8. Regime Translation Rules

8.1 Purpose

Regime translation rules map substrate variables into approximate descriptions used by known physics. They are not derivations yet. They are controlled dictionaries that say what must be shown in order for UST to earn correspondence with a known domain.

8.2 Geometry Translation

In the weak, stable, low-load regime, recoverability gradients may translate into curvature-like behaviour. Update pacing may translate into time. Compatibility may translate into effective locality. The goal is not to rename curvature as recoverability, but to show whether recoverability gradients reproduce the constraints that geometry normally captures.

$$\text{low } l, \text{ high } r, \text{ stable } C \Rightarrow \text{metric-like locality and weak-field geometry}$$

8.3 Quantum Translation

Quantum-like behaviour is translated through coherence capacity, envelope stability, compatibility topology, and collapse failure. Coherence capacity maps to the ability to sustain superposition-like multi-allocation. Compatibility maps to entanglement-like non-geometric correlation. Collapse maps to failure of recoverability or envelope viability under measurement-like disturbance.

high q, coherent E, stable C_E => envelope-supported quantum-like regime

8.4 Strong-Field Translation

Strong-field behaviour is translated through saturation, memory burden, recoverability collapse, and basin entry. The replacement target is not to deny black-hole observations. It is to determine whether the singularity is better represented as failure of geometric translation inside a deformity basin.

high l, high m, falling r, unstable C => saturation and basin-directed dynamics

9. Simulation Requirements

9.1 Minimal Simulation Engine

The first simulation engine should not attempt full physics. It should implement state vector arrays or graph nodes, compatibility-weighted adjacency, update operators, memory accumulation, recoverability evolution, loading redistribution, threshold detection, and envelope construction. The value of the engine is not spectacle. It is reproducibility.

- Initialise a finite set of substrate sites or relational graph nodes.
- Assign state vectors using logged seeds and parameter files.
- Compute compatibility weights at every tick.
- Update memory, loading, recoverability, coherence, compatibility, and organisation in frozen order.
- Detect threshold crossings and record regime transitions.
- Construct envelopes from compatibility and coherence conditions.
- Export complete run metadata for replication.

9.2 Output Metrics

The minimal output metrics should include overlap, coherence, memory load, recoverability, loading, envelope count, envelope lifetime, threshold distance, collapse-direction score, and reproducibility hash. These outputs are sufficient to evaluate whether the toy system produces stable regimes, hysteresis, saturation, or basin behaviour.

9.3 Validation Rules

The simulation must be deterministic under fixed seeds unless stochasticity is explicitly enabled and logged. Null models must be run against the same graph and initial conditions. Parameter sweeps must identify whether observed regimes are robust or artefacts of chosen constants. Every figure and score must be reproducible from an archived parameter file.

10. Open Problems and Unknowns

10.1 Unknown Functions

The exact recoverability function R is unknown. The exact threshold functions F_k are unknown. The exact compatibility weights are unknown. The correct memory kernel may not be exponential. The correct organisation variable may not be scalar or categorical. These unknowns are not weaknesses if they are recorded honestly. They become weaknesses only if the theory hides them.

10.2 Mathematical Gaps

The largest mathematical gaps are continuum limits, stability proofs, convergence conditions, existence of invariant regimes, and correspondence criteria. A discrete model can be useful without a continuum limit, but it cannot yet claim physical adequacy. The continuum programme must show when and why metric-like behaviour appears from relational update structure.

10.3 Simulation Gaps

The main simulation gaps are scaling, noise behaviour, boundary sensitivity, envelope detection, parameter identifiability, and visualisation of high-dimensional threshold surfaces. The first engine should therefore be small, inspectable, and deliberately boring. A fragile spectacular simulation is less useful than a modest reproducible one.

11. Final Statement

This document defines the first formal scaffold for UST: a minimal substrate state vector, a family of update operators, a compatibility metric, a memory kernel, a recoverability function, a loading rule, threshold surfaces, regime translation rules, and simulation requirements. It does not complete the theory. It gives the theory a place where it can become testable.

The central achievement is not mathematical finality. It is constraint. UST now has a minimal algebra that can be frozen, implemented, criticised, and improved. From this point forward, progress should be judged by whether the variables produce stable regimes, whether thresholds behave coherently, whether memory generates irreversible structure, whether recoverability explains transition behaviour, and whether any controlled correspondence to known physics can be earned rather than assumed.

Platinum Document #1 anchors the formalisation programme by converting UST's core language into simulation-ready objects. Its success depends on disciplined implementation, honest failure analysis, and refusal to mistake architectural elegance for completed physics.